

Types on paper: an email, counted and matched

Programming Paradigms

Session 3 — warm-up, before any code is written

Exercises 1–4: in class, in pairs, about 15 minutes. Exercises 5–6: homework.

SOLUTIONS

The lecture said that an algebraic data type is a closed list of shapes, written where the compiler can read it, and that the compiler pays us back by checking every **match**. We try that on the most ordinary object we own, an email message, with pen and paper. There is nothing here to run. The notation is OCaml's, and these are all we need:

<code>type t = A B C</code>	a <i>sum</i> : a value of <code>t</code> is A, or B, or C
<code>type t = A of int B of string</code>	a sum whose shapes carry something
<code>type t = { f : int; g : bool }</code>	a <i>product</i> : one value holds an <code>f</code> and a <code>g</code>
<code>int * bool</code>	a product without names, a pair
<code>t list, t option</code>	zero or more <code>t</code> ; a <code>t</code> or nothing (<code>Some x / None</code>)

1. Writing the type

Exercise 1 (the message)

Write OCaml types for the following, one line each, choosing *sum* or *product* as the facts demand. An **address** is a user name and a domain, both text. A **recipient** is an address that is either a *To*, a *Cc* or a *Bcc*. A **header** has a sender, a list of recipients, a subject, a date (take **date** as given) and an optional reply-to address. A **message** is a header and a body of text. Then draw one concrete message with two recipients as a tree: constructors at the nodes, text at the leaves.

Answer.

```
type address   = { user : string; domain : string }
type recipient = To of address | Cc of address | Bcc of address
type header    = { from : address; recipients : recipient list;
                  subject : string; date : date; reply_to : address option }
type message   = { header : header; body : string }
```

Address and header are products (every field is always present); recipient is a sum (exactly one of three kinds, each carrying an address). The reply-to is **address option**, not an address that may be the empty string: the type says it can be absent, and every function that reads it has to face that case. The tree has **message** at the root, **header** and the body text below it, and under **recipients** a list whose two elements are, say, *To* {...} and *Cc* {...}.

2. Counting values

Exercise 2 (why “algebraic”)

Let `type priority = Low | Normal | High`. How many distinct values are there of each of the following types? `bool * bool`; `priority`; `priority * bool`; `priority option`; `bool option option`; `(priority * bool) option`. State the two rules that give the answers.

Answer. 4; 3; 6; 4; 3; 7. A *sum* of types has as many values as its summands added: **option** adds one (`None`) to whatever it wraps, so `priority option` has 3 + 1, and `bool option option`

has $(2 + 1) + 1$. A *product* multiplies: `priority * bool` has 3×2 , and wrapping it in `option` gives $6 + 1$. That is the algebra in the name: constructors without payload are 1, `|` is $+$, $*$ and record fields are $\times$. (A recursive type, or one containing `string`, has infinitely many values — the rules still say how they are built.)

3. What the compiler will say

Exercise 3 (missing and unreachable)

Two functions on `recipient`, written in a hurry:

```
let address_of r = match r with
| To a -> a
| Cc a -> a

let is_hidden r = match r with
| To _ | Cc _ | Bcc _ -> false
| Bcc _ -> true
```

For each: does it compile? What does the compiler print, and what is the message called? What does `address_of (Bcc a)` do at run time, and what does `is_hidden (Bcc a)` return? Fix both.

Answer. Both compile — with a warning each, and the two warnings are the two dual checks of the lecture. The first is *Warning 8: this pattern-matching is not exhaustive. Here is an example of a case that is not matched: `Bcc _`*. At run time `address_of (Bcc a)` raises `Match_failure`. The second is *Warning 11: this match case is unused, on the arm `Bcc _ -> true`*: the first arm already takes every `Bcc`, so `is_hidden (Bcc a)` returns `false` — no crash, just the wrong answer, and only the warning tells us. Fixes: add `| Bcc a -> a`; and move `Bcc _ -> true` above the or-pattern, or take `Bcc _` out of it. Order matters because the first arm that fits wins.

4. States

Exercise 4 (a message's life)

A message is a draft, or has been sent at some date, or has been received at some date from some address, or has bounced with a reason. Write `type state`. Then explain why this is better than adding three fields to the header — `sent_at : date option`, `received_from : address option`, `bounce_reason : string option` — by naming one value the record version allows and the sum version cannot express.

Answer.

```
type state =
| Draft
| Sent of date
| Received of date * address
| Bounced of string
```

With three option fields, the record can hold a message that has a `sent_at` and a `bounce_reason`, or a `received_from` with no date, or none of the three while not being a draft: $2 \times 2 \times 2$ combinations, most of them meaningless, and every function must decide what they mean. The sum has exactly four shapes, each carrying only what that state has. A state that cannot be written cannot be mishandled.

5. A thread is a tree (homework)

Exercise 5 (replies)

A thread is a message together with the replies to it, each of which is itself a thread: *type thread = Msg of message * thread list*. Write, on paper, *count : thread -> int*, the number of messages in a thread. It needs a second function on *thread list*; write both, and say why each recursive call is on something smaller.

Answer.

```
let rec count (Msg (_, replies)) = 1 + count_all replies
and count_all ts = match ts with
| [] -> 0
| t :: rest -> count t + count_all rest
```

Two functions, one per type, declared together with **and** because each calls the other. **count** calls **count_all** on the list of replies, which is a part of the thread; **count_all** calls **count** on one reply, a strictly smaller thread, and itself on a shorter list. Every call descends, so it ends. The same pair appears whenever a type holds a list of itself.

6. Kinds and operations (homework)

Exercise 6 (the table)

Three operations on *recipient*: *show* (as text), *deliver* (send to its address), *is_hidden*. Draw the table with the three kinds *To*, *Cc*, *Bcc* as rows and the operations as columns. In OCaml, which cells sit together in one function? Now add a kind, *Bcc_group of address list*, and separately add an operation, *count*: for each change, which cells are new and how many existing functions are touched? Sketch the same type as a Java abstract class with three subclasses and answer the same two questions.

Answer. In OCaml a *column* is one function: **show** is one **match** holding the *To*, *Cc* and *Bcc* cells. Adding **count** adds one column — one new function, no existing function touched. Adding **Bcc_group** adds a row — three new cells, one in each existing function, and the compiler lists all three (Warning 8). In Java a *row* is one class: *To* holds its own **show**, **deliver** and **isHidden**. Adding the kind is one new class, nothing else touched. Adding **count** is a new abstract method and one body in each of the three classes; the compiler refuses to build until all three exist. Each way makes one direction free and the other expensive, and in both the compiler checks the expensive direction. That is the expression problem, and session 10 is about the ways out of it.